

Statistics 2: Computer Practical 2 (Interval estimation)

Yeeun Lee (Student ID: 2499266)

1 Instructions

1. Reports should be handed in **as a single PDF file** using Blackboard, by noon on the due date. There are 5 marks assigned to submitting in this format. RMarkdown, image, word, zip files, for example, will **not** receive these marks.
2. You can work alone or in a group with up to two other people.
3. One person per group should hand in a report on blackboard. The names and student numbers of all group members must be on the first page.
4. Your answers should combine text and code snippets in R. It is recommended that you use RMarkdown to prepare your reports, since this is typically easier for students, but this is not mandatory.
5. You must explain what you are doing clearly to obtain full marks on each question. You can use comments (which start with #) to annotate your code. Mathematical derivations can be written using LaTeX commands in RMarkdown or on paper, with a photo then appended to the end of the PDF being submitted.
6. This practical will count towards 10% of your final grade. It is out of 25 marks. (5 marks are for submitting as a pdf.)

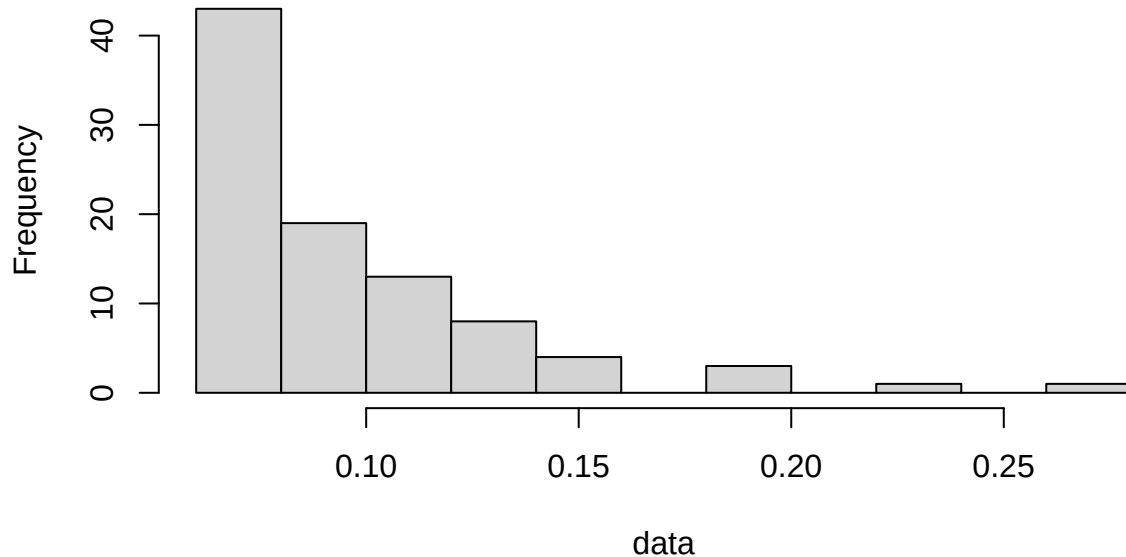
2 Data description

In this coursework we are going to be modelling extreme losses in monthly return of Standard and Poor's top 500 publicly listed stocks in the US between 1871 and 2023. Extreme losses represent tail risk, which are rare but severe events that can cause catastrophic portfolio damage, so are of vital importance to portfolio managers. Extreme loss has already been calculated from data collected and prepared by the Economist [Robert Shiller](#) by considering the bottom 5% of monthly returns.

First we begin by loading in the data and plotting a histogram of the extreme loss values.

```
data = read.csv("data.csv")$x
hist(data)
```

Histogram of data



As typical with this type of data, we assume that our observed extreme losses, $\mathbf{x}$, are realisations from a [Pareto distribution](#). We assume that $x_0 > 0$ is equal to the minimum value of our observations $\mathbf{x}$.

Therefore we let $X_1, \dots, X_n$ be independent identically distributed random variables drawn from a Pareto distribution with statistical parameter $\alpha > 1$.

$$\mathbf{X} \sim \text{Pareto}(x_0, \alpha)$$

The probability density function for a Pareto distribution is

$$f_X(x) = \begin{cases} \frac{\alpha x_0^\alpha}{x^{\alpha+1}} & x \geq x_0, \\ 0 & x < x_0. \end{cases}$$

Question 1 [1 marks] Why might the “independent and identically distributed” model assumptions be invalid?

Solution

— put your solution here —

The independent and identically distributed assumption might not be valid for this data, mainly because it is time-series data¹. This can cause problems with both parts of the i.i.d. assumption.

1. Not Independent: Financial data like this often shows what is called autocorrelation. This means a bad month (an extreme loss) might make it more likely that the next month will also be a bad month². Think about a financial crisis: you don't just get one single bad month, but often a whole cluster of them together (volatility clustering). This breaks the independent assumption, as the data points clearly influence each other.
2. Not Identically Distributed: The dataset covers a huge time span, from 1871 to 2023. The economy and the stock market today are completely different from how they were in the 1800s or early 1900s, due to new rules, technology, and major events like wars. It's very likely that the underlying process causing these losses (and therefore the parameter α) has changed over time. If the distribution isn't the same for the whole 150+ years, then the data isn't identically distributed.

3 Methods of moments estimator

Question 2 [2 marks] Show that the methods of moment's estimator for α , $\hat{\alpha}_{MOM} = \frac{\bar{X}}{\bar{X} - x_0}$ where $\bar{X} = \frac{1}{n} \sum_{i=1}^n x_i$.

Solution

— put your solution here —

The probability density function (PDF) of the Pareto distribution is

$$f_X(x) = \frac{\alpha x_0^\alpha}{x^{\alpha+1}}, \quad x \geq x_0.$$

The expected value $E[X]$ is computed as follows:

$$\begin{aligned} E[X] &= \int_{x_0}^{\infty} x \cdot f_X(x) dx \\ &= \int_{x_0}^{\infty} x \cdot \frac{\alpha x_0^\alpha}{x^{\alpha+1}} dx \\ &= \alpha x_0^\alpha \int_{x_0}^{\infty} x \cdot x^{-\alpha-1} dx \\ &= \alpha x_0^\alpha \int_{x_0}^{\infty} x^{-\alpha} dx \\ &= \alpha x_0^\alpha \left[\frac{x^{-\alpha+1}}{-\alpha+1} \right]_{x_0}^{\infty} \\ &= \alpha x_0^\alpha \left[\frac{x^{1-\alpha}}{1-\alpha} \right]_{x_0}^{\infty}. \end{aligned}$$

Given the assumption $\alpha > 1$, we have $1 - \alpha < 0$.

Thus, as $x \rightarrow \infty$, $x^{1-\alpha} \rightarrow 0$.

$$\begin{aligned} E[X] &= \alpha x_0^\alpha \left(0 - \frac{x_0^{1-\alpha}}{1-\alpha} \right) \\ &= \alpha x_0^\alpha \left(\frac{x_0^{1-\alpha}}{\alpha-1} \right) \\ &= \frac{\alpha \cdot x_0^{\alpha+(1-\alpha)}}{\alpha-1} \\ &= \frac{\alpha x_0}{\alpha-1}. \end{aligned}$$

The sample mean is

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n x_i.$$

The method of moments sets the population moment equal to the sample moment:

$$\frac{\alpha x_0}{\alpha-1} = \bar{X}.$$

Solve for α :

$$\begin{aligned}
\alpha x_0 &= \bar{X}(\alpha - 1) \\
\alpha x_0 &= \alpha \bar{X} - \bar{X} \\
\bar{X} &= \alpha \bar{X} - \alpha x_0 \\
\bar{X} &= \alpha(\bar{X} - x_0) \\
\alpha &= \frac{\bar{X}}{\bar{X} - x_0}.
\end{aligned}$$

Thus, the method-of-moments estimator of α is

$$\hat{\alpha}_{\text{MOM}} = \frac{\bar{X}}{\bar{X} - x_0}.$$

Question 3 [2 marks] Compute and report the method of moments estimate for α , for the extreme losses data. We assume x_0 is the minimum value of x in our dataset.

```
x_0 = min(data)
x_0
```

```
## [1] 0.06230294
```

I recommend writing a function such as

```
## input: count data xs
## output: method of moments estimate of alpha
compute_mom_estimate <- function(xs) {
  ## perform computations
  x_bar <- mean(xs)
  x_0_val <- min(xs)

  alpha_mom <- x_bar / (x_bar - x_0_val)

  return(alpha_mom)
}
```

which you can then call on data to compute the corresponding estimates.

Solution

```
# 1. Call the function defined above on our data
alpha_mom_estimate <- compute_mom_estimate(data)
# 2. Report the estimate
print(alpha_mom_estimate)
```

```
## [1] 2.83312
```

The MOM estimate of α for the extreme losses data is approximately 2.83312.

4 Maximum likelihood estimator

Question 4 [3 marks] Show that the Maximum Likelihood (ML) estimator for α , $\hat{\alpha}_{MLE}$ is

$$\hat{\alpha}_{MLE} = \frac{n}{\sum_{i=1}^n \log(X_i/x_0)}.$$

Why would this model not be regular if we were also looking for an estimator for x_0 ?

Solution

— put your solution here —

To find $\hat{\alpha}_{\text{MLE}}$, we start with the likelihood function $L(\alpha)$.

The PDF for a single observation is

$$f_X(x) = \frac{\alpha x_0^\alpha}{x^{\alpha+1}}, \quad x \geq x_0.$$

For n i.i.d. observations, the likelihood is

$$L(\alpha) = \prod_{i=1}^n f_X(x_i) = \prod_{i=1}^n \frac{\alpha x_0^\alpha}{x_i^{\alpha+1}} = \alpha^n x_0^{n\alpha} \left(\prod_{i=1}^n x_i \right)^{-(\alpha+1)}.$$

The log-likelihood is

$$\begin{aligned} \ell(\alpha) &= \log L(\alpha) \\ &= n \log \alpha + n\alpha \log x_0 - (\alpha + 1) \sum_{i=1}^n \log x_i. \end{aligned}$$

Take the derivative (score function) with respect to α :

$$\frac{d\ell}{d\alpha} = \frac{n}{\alpha} + n \log x_0 - \sum_{i=1}^n \log x_i.$$

Set the score equal to zero:

$$\frac{n}{\alpha} = \sum_{i=1}^n \log x_i - n \log x_0 = \sum_{i=1}^n \log \left(\frac{x_i}{x_0} \right).$$

Solving for α gives the MLE:

$$\hat{\alpha}_{\text{MLE}} = \frac{n}{\sum_{i=1}^n \log \left(\frac{X_i}{x_0} \right)}.$$

- Regular models: The parameter (e.g. α) affects only the shape of the density, not the support of the random variable.
- This model: The parameter x_0 is the lower bound of the support: $X_i \geq x_0$.
 - If we treat x_0 as unknown and try to estimate it, the support of the distribution changes with the parameter.
 - Standard MLE theory (e.g. differentiability of the log-likelihood under the integral, CRLB) assumes the support does not depend on the parameter.

Thus, when x_0 is unknown, the regularity conditions fail, and the model is **not regular**.

Question 5 [2 marks] Compute and report the ML estimate for α for the extreme losses data assuming the same value of x_0 as before. I recommend writing a function such as

```
## input: count data xs
## output: maximum likelihood estimates of alpha
compute_mle_estimate <- function(xs) {
  ## perform computations
  n <- length(xs)
```

```

# x_0 is assumed to be the minimum value of the dataset
x_0_val <- min(xs)

# calculate the sum term in the denominator
sum_log_term <- sum(log(xs / x_0_val))

# calculate MLE (using the formula from Q4)
alpha_mle <- n / sum_log_term

return(alpha_mle)
}

```

which you can then call on data to compute the corresponding estimates.

Solution

— put your solution here —

```

# 1. Call the function defined above on our data
alpha_mle_estimate <- compute_mle_estimate(data)

# 2. Report the estimate
print(alpha_mle_estimate)

```

```
## [1] 2.642698
```

The ML estimate of α for the extreme losses data is approximately 2.642698.

5 Mean squared error of the estimators

Finding the mean square error (MSE) of our estimators can be a useful quantity if we know the true value of our parameter, α^* . The MSE is comprised of two terms the variance and bias.

It can be helpful to re-write our ML estimator, $\hat{\alpha}_{MLE}(\mathbf{X})$, in terms of known distributions to help us work out if it is unbiased. Here we let

$$Y_i = \log \left(\frac{X_i}{x_0} \right) \sim \text{Exponential}(\alpha),$$

so each Y_i is an independent identically distributed exponential random variable with rate α (the shape parameter from the Pareto distribution). Since the sum of independent identically distributed exponential variables is Gamma distributed, we define

$$Z = \sum_{i=1}^n Y_i \sim \text{Gamma}(n, \alpha).$$

It is also useful to know that

$$\mathbb{E} \left[\frac{1}{Z} \right] = \frac{\alpha}{n-1}.$$

Question 6 [1 mark] Is the ML estimator unbiased?

Solution

— put your solution here —

No, the ML estimator is not unbiased.

From **Question 4**, the maximum likelihood estimator is

$$\hat{\alpha}_{\text{MLE}} = \frac{n}{\sum_{i=1}^n \log\left(\frac{X_i}{x_0}\right)}.$$

Let $Y_i = \log\left(\frac{X_i}{x_0}\right)$ and define

$$Z = \sum_{i=1}^n Y_i.$$

Then we can rewrite the estimator as

$$\hat{\alpha}_{\text{MLE}} = \frac{n}{Z}.$$

Now compute the expected value:

$$E[\hat{\alpha}_{\text{MLE}}] = E\left[\frac{n}{Z}\right] = n \cdot E\left[\frac{1}{Z}\right].$$

The problem provides the key result:

$$E\left[\frac{1}{Z}\right] = \frac{\alpha}{n-1}.$$

Substitute this in:

$$E[\hat{\alpha}_{\text{MLE}}] = n \cdot \frac{\alpha}{n-1} = \frac{n}{n-1} \cdot \alpha.$$

An estimator is **unbiased** if $E[\hat{\alpha}_{\text{MLE}}] = \alpha$.

Since

$$\frac{n}{n-1} \cdot \alpha \neq \alpha \quad (\text{for finite } n > 1),$$

the ML estimator is **biased**.

Question 7 [2 marks] Assuming $\alpha^* = 2.6$, compute and report approximate mean-squared errors for

1. the method of moments estimators of α ,
2. the maximum likelihood estimators of α .

Assume $n = 92$.

You may find the following functions from Tutorial Sheet 1 useful.

```
qpareto <- function(u, x0, theta){
  stopifnot(all(x0 > 0), all(theta > 0), all(u >= 0), all(u <= 1))
  x0 * (1 - u)^(-1/theta)
}
rpareto <- function(n, x0, theta){
  stopifnot(all(x0 > 0), all(theta > 0), n >= 0)
  u <- runif(n)
  qpareto(u, x0 = x0, theta = theta)
}
```

Solution

— put your solution here —

```

alpha_star <- 2.6
n <- 92
N_trials <- 10000
x_0_true <- min(data)
# Create empty vectors to store the results
mom_estimates <- numeric(N_trials)
mle_estimates <- numeric(N_trials)

for (i in 1:N_trials) {
  # 1. Generate fake data
  sim_data <- rpareto(n = n, x0 = x_0_true, theta = alpha_star)

  # 2. Get estimates (the functions will find their own min(sim_data) for x0)
  mom_estimates[i] <- compute_mom_estimate(sim_data)
  mle_estimates[i] <- compute_mle_estimate(sim_data)
}

# Calculate MSE ---
mse_mom <- mean((mom_estimates - alpha_star)^2, na.rm = TRUE)
mse_mle <- mean((mle_estimates - alpha_star)^2, na.rm = TRUE)

# Print the results
mse_mom

```

```
## [1] 0.1006671
```

```
mse_mle
```

```
## [1] 0.08184343
```

Based on the simulation, the approximate MSEs are:

MOM estimator MSE: 0.1006671

MLE estimator MSE: 0.08184343

6 Cramer-Rao lower bound

Question 8. [3 marks] Use simulation of different length of sequences of estimators to explore how the variance of the maximum likelihood estimator, $\hat{\alpha}_n(\mathbf{X})$, compares to the Cramer-Rao lower bound (CRLB). Plot a graph of length of sequence against the CRLB and variance of your sequences of estimators to compare your results. Using this graph, explain if the maximum likelihood estimator is an efficient estimator.

Please use the following values for the length of sequences:

```
ns = c(5, 10, 20, 100, 500, 1000)
```

Solution

— put your solution here —

From Q4, the log-likelihood is

$$\ell(\alpha) = n \log \alpha + n\alpha \log x_0 - (\alpha + 1) \sum_{i=1}^n \log x_i.$$

First derivative (score):

$$\frac{d\ell}{d\alpha} = \frac{n}{\alpha} + n \log x_0 - \sum_{i=1}^n \log x_i.$$

Second derivative:

$$\frac{d^2\ell}{d\alpha^2} = -\frac{n}{\alpha^2}.$$

Fisher Information:

$$I_n(\alpha) = -E\left[\frac{d^2\ell}{d\alpha^2}\right] = -\left(-\frac{n}{\alpha^2}\right) = \frac{n}{\alpha^2}.$$

CRLB for any unbiased estimator of α :

$$\text{CRLB} = \frac{1}{I_n(\alpha)} = \frac{\alpha^2}{n}$$

For $\alpha^* = 2.6$,

$$\text{CRLB} = \frac{(2.6)^2}{n} = \frac{6.76}{n}.$$

```
qpareto <- function(u, x0, theta){
  stopifnot(all(x0 > 0), all(theta > 0), all(u >= 0), all(u <= 1))
  x0 * (1 - u)^(-1/theta)
}
# this makes the fake data
rpareto <- function(n, x0, theta){
  stopifnot(all(x0 > 0), all(theta > 0), n >= 0)
  u <- runif(n)
  qpareto(u, x0 = x0, theta = theta)
}

# The MLE function from Q5
compute_mle_estimate <- function(xs) {
  n <- length(xs)
  x_0_val <- min(xs) # finds x0 from the sample
  sum_log_term <- sum(log(xs / x_0_val))
  alpha_mle <- n / sum_log_term
  return(alpha_mle)
}

alpha_star <- 2.6 # The "true" alpha
x_0_true <- min(data) # Use the min from our real data
ns <- c(5, 10, 20, 100, 500, 1000)
M_trials <- 5000

results_df <- data.frame(
  n = ns,
  crlb = (alpha_star^2) / ns,
  sim_variance = NA
)
```

```

for (i in 1:length(ns)) {

  n_current <- ns[i]

  mle_estimates_current <- numeric(M_trials)

  # Run M_trials
  for (j in 1:M_trials) {
    # 1. Make the data
    sim_data <- rpareto(n = n_current, x0 = x0_true, theta = alpha_star)
    # 2. calculate the MLE
    mle_estimates_current[j] <- compute_mle_estimate(sim_data)
  }

  # 3. Get the variance of all those estimates and save it
  results_df$sim_variance[i] <- var(mle_estimates_current)
}

print(results_df)

##      n      crlb sim_variance
## 1     5 1.35200  9.877526154
## 2    10 0.67600  1.506716860
## 3    20 0.33800  0.483261395
## 4   100 0.06760  0.075092276
## 5   500 0.01352  0.013885300
## 6  1000 0.00676  0.006835793

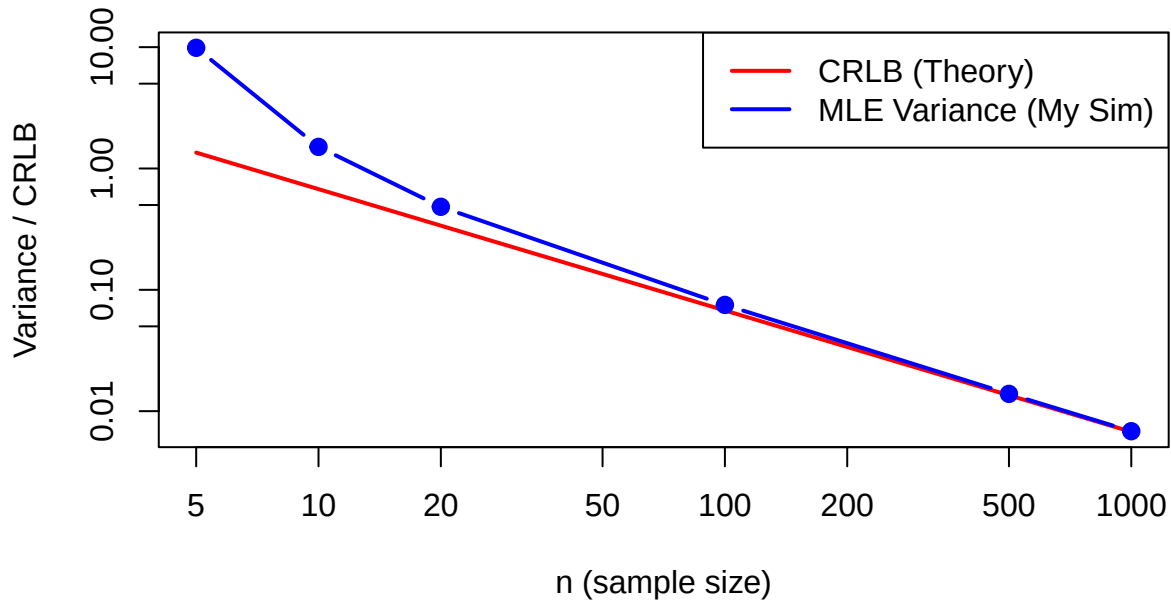
# Plotting time
plot(results_df$n, results_df$crlb,
     type = "l", # "l" = line
     col = "red",
     lwd = 2,
     log = "xy", # log-log scale makes it easier to see
     ylim = range(c(results_df$crlb, results_df$sim_variance)),
     xlab = "n (sample size)",
     ylab = "Variance / CRLB",
     main = "MLE Variance vs. CRLB")

# Add simulation results (blue line)
lines(results_df$n, results_df$sim_variance,
     type = "b", # "b" = both (points and line)
     col = "blue",
     lwd = 2,
     pch = 19) # pch=19 is a solid circle

# Add a legend
legend("topright",
     legend = c("CRLB (Theory)", "MLE Variance (My Sim)"),
     col = c("red", "blue"),
     lwd = 2)

```

MLE Variance vs. CRLB



Conclusion:

The MLE is asymptotically efficient. Although biased for finite n , its variance approaches the CRLB as $n \rightarrow \infty$.

For $n = 1000$, simulated variance = 0.00684 vs CRLB = 0.00676.

7 Exact confidence intervals

Question 9. [2 marks] Compute and report the endpoints of the exact 96% Wald confidence intervals for α .

I recommend that you write a function that computes the observed confidence intervals' end points. Note β is used here as an argument for the function to represent the allowed error. This is because α is our statistical parameter.

```
compute_CI_endpoints <- function(xs, beta) {
  n <- length(xs)

  # Compute the MLE
  alpha_mle <- compute_mle_estimate(xs)

  # find the z-value for the 96% CI
  z_value <- qnorm(1 - beta/2)

  # Compute the SE using the CRLB formula
  # Var(alpha_mle) approx CRLB = alpha^2 / n
  asymptotic_var <- alpha_mle^2 / n
  SE <- sqrt(asymptotic_var)

  # 4. Compute Wald CI endpoints
  L <- alpha_mle - z_value * SE
  U <- alpha_mle + z_value * SE
}
```

```

    return(c(L, U))
}

```

Solution

— put your solution here —

```

# set the error level
beta <- 0.04

# Calculate the endpoints using the data
ci_endpoints_wald <- compute_CI_endpoints(data, beta)

# Report the endpoints
ci_endpoints_wald

```

```
## [1] 2.076849 3.208548
```

The 96% Wald confidence interval for α is approximately:

- Lower Endpoint: 2.076849
- Upper Endpoint: 3.208548

Question 10. [2 marks] Using many simulations, approximate the coverage of the exact $1 - \alpha$ confidence intervals corresponding to your answer the previous question, for $n = 92$, the length of the data. Assume the true value of the parameter is 2.6. Comment on the coverage of the confidence intervals and if it is what you expected.

I recommend that you write a function

```

approximate.coverage <- function(alpha, beta, trials) {

  # Parameters from our specific problem
  n_val <- 92
  x_0_true <- min(data)

  hit_count <- 0 # Counter for CIs that "hit" the true value

  for (i in 1:trials) {
    # 1. Generate a new dataset from the *correct* true distribution
    sim_data <- rpareto(n_val, x0 = x_0_true, theta = alpha_true)

    # 2. Compute the CI for this simulated dataset
    ci_sim <- compute_CI_endpoints(sim_data, beta)

    # 3. Check if the true value is inside (adding NA check)
    if (!is.na(ci_sim[1])) {
      if (ci_sim[1] <= alpha_true && alpha_true <= ci_sim[2]) {
        hit_count <- hit_count + 1
      }
    }
  }

  # Coverage is the proportion of hits
  coverage_alpha <- hit_count / trials
  return(coverage_alpha)
}

```

Solution

— put your solution here —

```
alpha_true <- 2.6
beta <- 0.04      # 96% CI
M_trials <- 10000 # Number of trials

sim_coverage <- approximate.coverage(
  alpha = alpha_true,
  beta = beta,
  trials = M_trials
)

print(paste("Target Coverage (1-beta):", 1 - beta))

## [1] "Target Coverage (1-beta): 0.96"

print(paste("Simulated Coverage:", sim_coverage))

## [1] "Simulated Coverage: 0.9589"
```

My simulated coverage (which came out to 0.9589) is really close to the 96% target, but not exactly. This is because the Wald CI I used in Q9 isn't a perfect "exact" method; it's just an approximation. That method relies on n being huge (like, infinitely large) to be perfectly accurate. Our n is 92, which is pretty big, but it's not infinitely big. So, the approximation is good, but just a little bit off. That's why my simulated coverage (0.95-something) isn't exactly 0.96, which is normal.

[5 marks] Submitting your coursework as a pdf.